# ToddySoft Connect S7 Driver - Ignition 8.3

Ignition 8.3 module for connecting to Siemens SIMATIC controllers — S7-300, S7-400, S7-1200, S7-1500 and family devices such as Simocode, Sinamics and LOGO! — over ISO-on-TCP. The driver speaks both classic S7Comm and S7CommPlus, the TLS-secured native protocol of the S7-1200 / S7-1500, and negotiates between them at connect time.

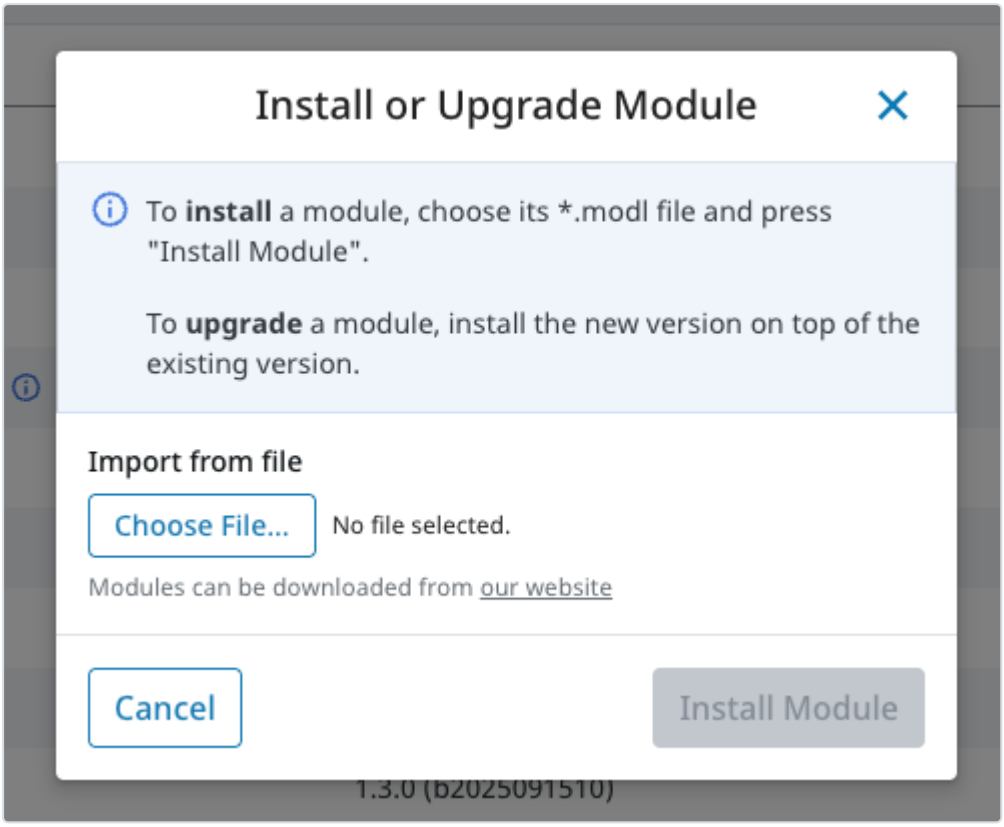
The device configuration uses a custom configuration form: settings that do not apply to what you have selected stay out of your way, and the fields that carry a security consequence say so where you read them.

> **Choosing between this module and the S7 Web API module.** This one is the process-data path: binary framing, batched reads, real subscriptions. The **S7 Web API** module reaches surfaces this driver cannot — the diagnostic buffer, alarms, the operating mode — over HTTPS, at roughly a tenth of the per-tag speed and with no change notification on the wire. Use this module to collect data, and the Web API module for diagnostics or where network policy permits only HTTPS.

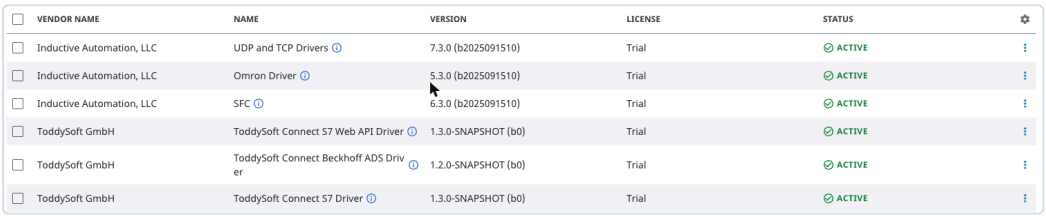
## Installation

> **This module is for Ignition 8.3 only.** It is built against the Ignition 8.3 SDK and will not run on any other major version. For Ignition 8.1, install the separate `…-8.1.modl` build instead — the two are not interchangeable, and each gateway needs the build that matches it.
>
> **Ignition 8.0 and earlier are not supported and cannot be.** The module is built on the device-driver API that Ignition introduced with 8.1, so on 8.0 the gateway fails to load it with `NoClassDefFoundError: … opcua.server.api.AbstractDeviceModuleHook`.
>
> Both builds carry the same name in the gateway's module list, so tell them apart by the file you installed or by the module description, which names the Ignition version.
>
> **Ignition's OPC UA module must be installed and running.** This is an OPC UA device driver: it registers with Ignition's own OPC UA server, and every tag it exposes is served through it. With that module absent the gateway cannot load this one, and the log fills with `ClassNotFoundException` / `NoClassDefFoundError` naming `com.inductiveautomation.opcua…` classes. The OPC UA module ships with Ignition and is installed by default, so in practice this bites only on a gateway where it has been removed or disabled. This module declares the dependency as **required** — a mechanism 8.3 has and 8.1 lacks — so Ignition does not attempt to load it at all, and reports the reason instead of failing with a stack trace. In **Status > Diagnostics > Logs** you will see:
>
> `Module "com.toddysoft.connect.…" depends on "com.inductiveautomation.opcua", which is either not installed or inactive on this gateway.`
>
> This is the same handling Ignition's own drivers get in the same situation.

1. Open the Ignition Gateway web interface and navigate to **Config > System > Modules**.
2. Click **Install or Upgrade a Module...**.
3. Select the `ToddySoft-Connect-S7-Driver-8.3.modl` file and upload it.



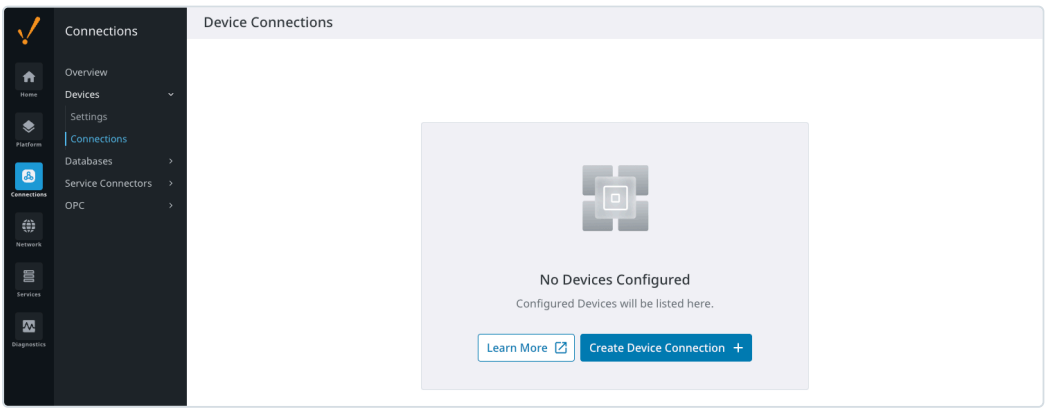

4. Accept the license / certificate prompt to continue the installation.
5. Wait for the Gateway to finish and verify the module appears as **Running** in the modules list.



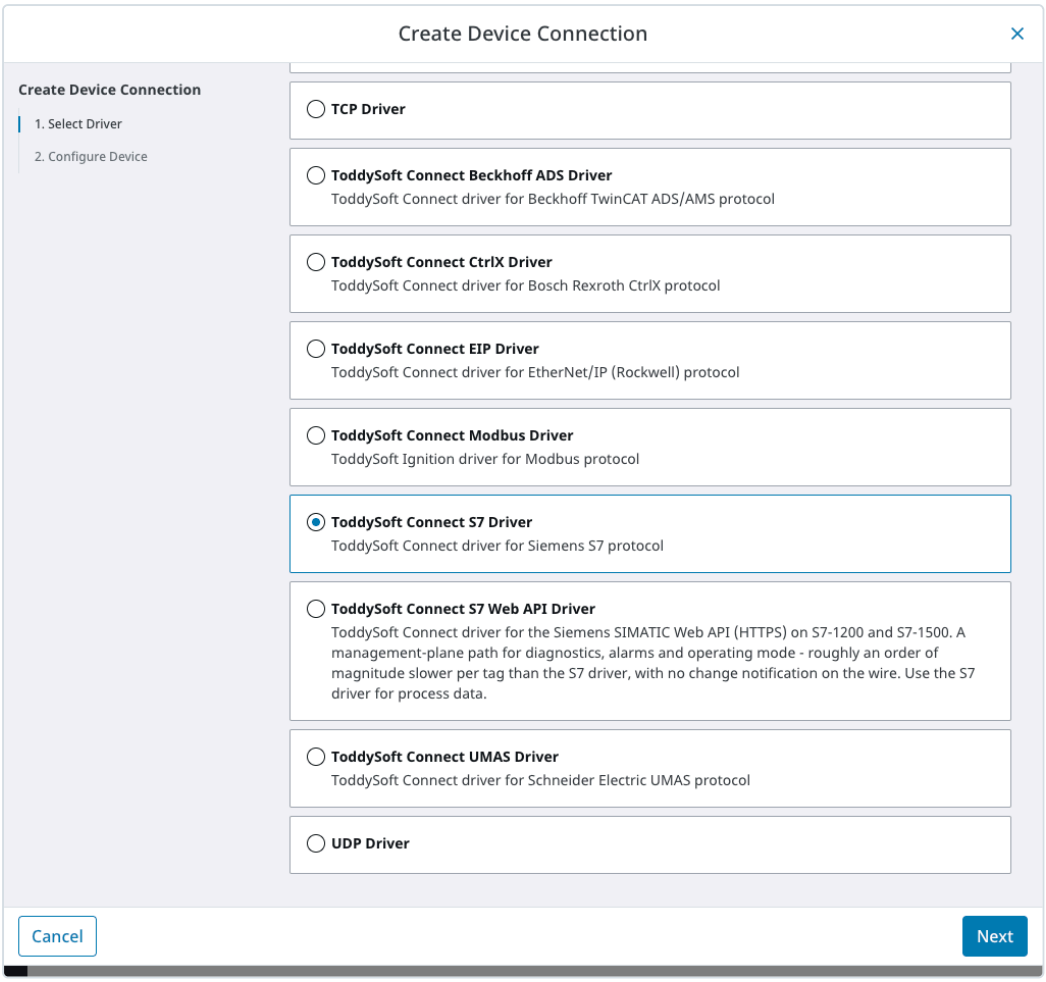
| VENDOR NAME | NAME | VERSION | LICENSE | STATUS |
|---|---|---|---|---|
| Inductive Automation, LLC | UDP and TCP Drivers | 7.3.0 (b2025091510) | Trial | ACTIVE |
| Inductive Automation, LLC | Omron Driver | 5.3.0 (b2025091510) | Trial | ACTIVE |
| Inductive Automation, LLC | SFC | 6.3.0 (b2025091510) | Trial | ACTIVE |
| ToddySoft GmbH | ToddySoft Connect S7 Web API Driver | 1.3.0-SNAPSHOT (b0) | Trial | ACTIVE |
| ToddySoft GmbH | ToddySoft Connect Beckhoff ADS Driver | 1.2.0-SNAPSHOT (b0) | Trial | ACTIVE |
| ToddySoft GmbH | ToddySoft Connect S7 Driver | 1.3.0-SNAPSHOT (b0) | Trial | ACTIVE |

## Creating a Device Connection

1. Go to **Config > OPC UA > Device Connections**.

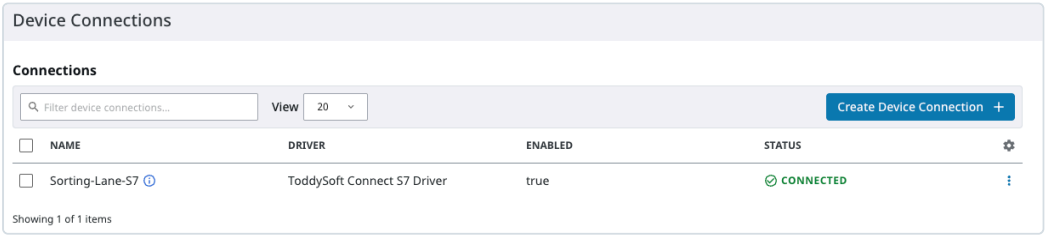


2. Click **Create new Device...** and select **ToddySoft Connect S7 Driver** as the device type.



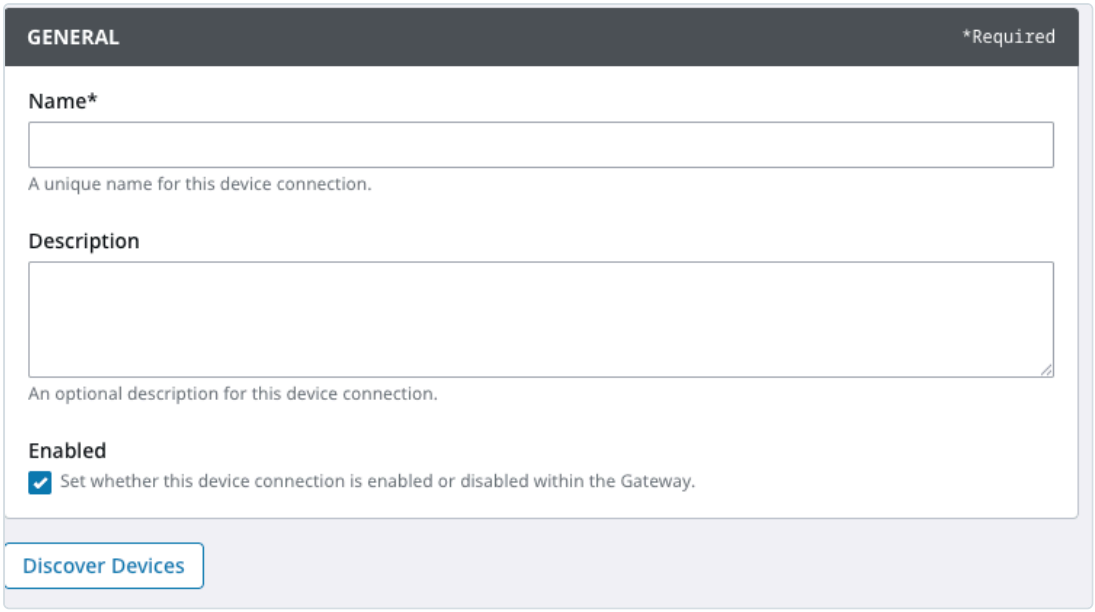

3. Configure the connection settings (see the sections below) and click **Create Device Connection**.

4. The new device then appears in the Device Connections list.




## Finding devices on the network

When you create a device, the configuration form carries a **Discover Devices** button directly beneath the **General** section, above the first settings card.

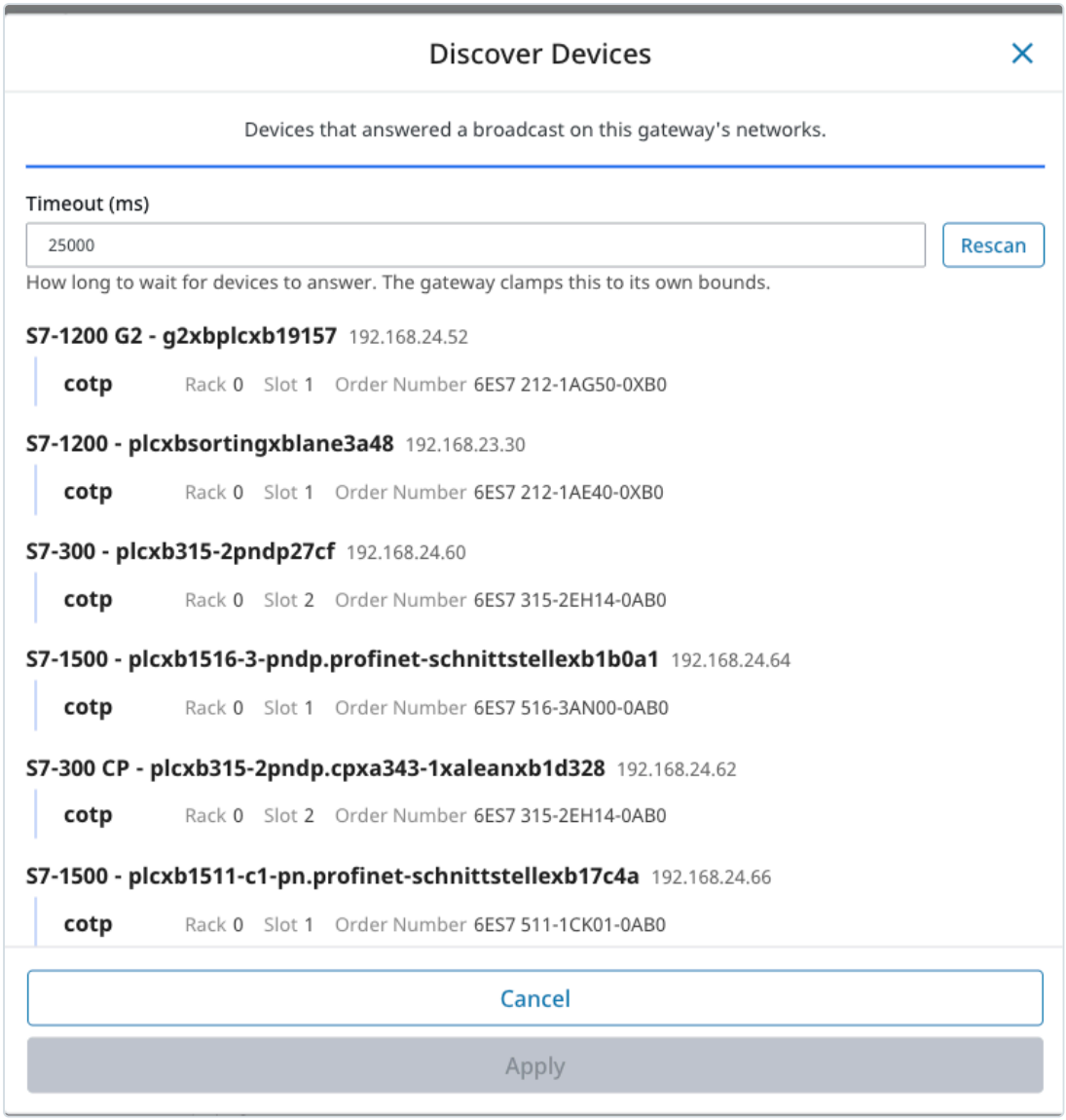



## Discover Devices

Sweeps the gateway's own networks with a PROFINET-DCP broadcast and lists the controllers that answer, each with the transport it offers and the **rack and slot** it reports. Those two numbers are most of the payoff: they are otherwise read off the physical rack and typed in by hand, and a wrong pair frequently connects — to a different CPU, which then answers reads with perfectly valid data from the wrong machine.

A result looks like this, one block per controller:

```
S7-1200 - plcxbsortingxblane3a48   192.168.23.30
   cotp    Rack 0   Slot 1   Order Number 6ES7 212-1AE40-0XB0
```




**Timeout (ms)** bounds how long the gateway listens. It defaults to **25000**, which is deliberate: a DCP sweep takes roughly fifteen seconds, plus about four more spent probing each device found for its order number and firmware. A shorter bound stops the sweep before the answers arrive and the panel then reports *No devices answered* over a network that is full of controllers. The gateway clamps the value to its own limits, currently one second to thirty.

Selecting a result fills in host, port, rack and slot. Three things it deliberately does not do:

- **It fills in only what the device actually reported.** A field the broadcast said nothing about is left alone rather than guessed at.
- **It never fills a password.** A broadcast tells you where a device is, never how to authenticate to it.
- **It refuses a result it cannot address.** Some controllers answer as a *named* endpoint — a remote TSAP string instead of a rack and slot. This driver has no field for a named endpoint, and defaulting to rack 0 / slot 1 would silently point the

device at a different CPU, so the result is rejected with a message saying so rather than applied.

If applying a result would overwrite something you have already typed, it lists what would change and asks first. Running a sweep needs permission to create device connections; being logged in is not enough.

> **In Docker, discovery needs the discovery agent.** A container sits on Docker's network, not on the plant network the controllers are on, so the search alone finds nothing there. See Running in Docker: the discovery agent; without it, enter the host, rack and slot by hand — everything else on this page works normally.

> **This driver offers no *Test Connection* button.** That action appears only for drivers that publish a device-setup capability the gateway can probe, and this one publishes none — so there is nothing it could check and nothing it could repair. The button is absent rather than present and inert. The Beckhoff ADS module does offer it.

## Running in Docker: the discovery agent

A gateway in a Docker container sees only the container's networks, not the plant's: the PROFINET-DCP broadcast behind *Discover Devices* never leaves the container's network. On top of that, Ignition's official Docker image carries no libpcap, which the search needs to send and capture its frames, so in a container it cannot run at all and reports that nothing answered. The **ToddySoft discovery agent** closes that gap. It is a small, separate container that runs on the same machine with the host's network and searches the plant networks on the gateway's behalf. The agent only searches: connecting, browsing and reading stay with the gateway, through the machine's normal routing. A gateway installed directly on the plant machine, or a container on the host network, does not need it.

**Installing it.** The agent comes as an image archive next to this module, in the share you download the module from: `discovery-agent-<version>-<architecture>.tar.gz`, for `amd64` or `arm64`. Use the agent of the module's version. On the machine that runs the Ignition container (Linux):

```
docker load < discovery-agent-1.3.0-amd64.tar.gz
docker run -d --name toddysoft-discovery-agent \
  --restart unless-stopped --network host --cap-add NET_RAW \
  toddysoft/discovery-agent:1.3.0
```

`--network host` lets it search the plant networks, and `NET_RAW` lets it send and capture the discovery frames. Docker Desktop (macOS, Windows) runs containers in a virtual machine, whose networks the agent would search instead of the plant's.

**Nothing to set in Ignition.** On every **Discover Devices** the module looks for the agent at its container's default gateway, which on Docker is the machine itself. An agent started later is found on the next search, with no restart. A line above the results says what happened:

- *via discovery agent*: the agent searched; its devices are listed together with any the gateway found itself, each once.
- *no discovery agent found · container networks only*: no agent runs; the search is the gateway's own, as before.
- *discovery agent refused: …* or *discovery agent not usable: …*: the reason follows, for example the licence or the shared secret below.

**The panel shows its results after about five seconds**, when the agent has finished. The devices listed come from the agent: the gateway's own search cannot run inside the container (see above), which the gateway log notes as *this gateway's own search could not run … the discovery agent answered*. Without an answering agent the panel says why under *Part of the search failed*.

**Licence.** The agent searches only for a gateway licensed for this driver. With each search the module sends the licence it runs on, the bundled trial included, over an encrypted connection; the agent checks it, uses it for that search and keeps nothing.

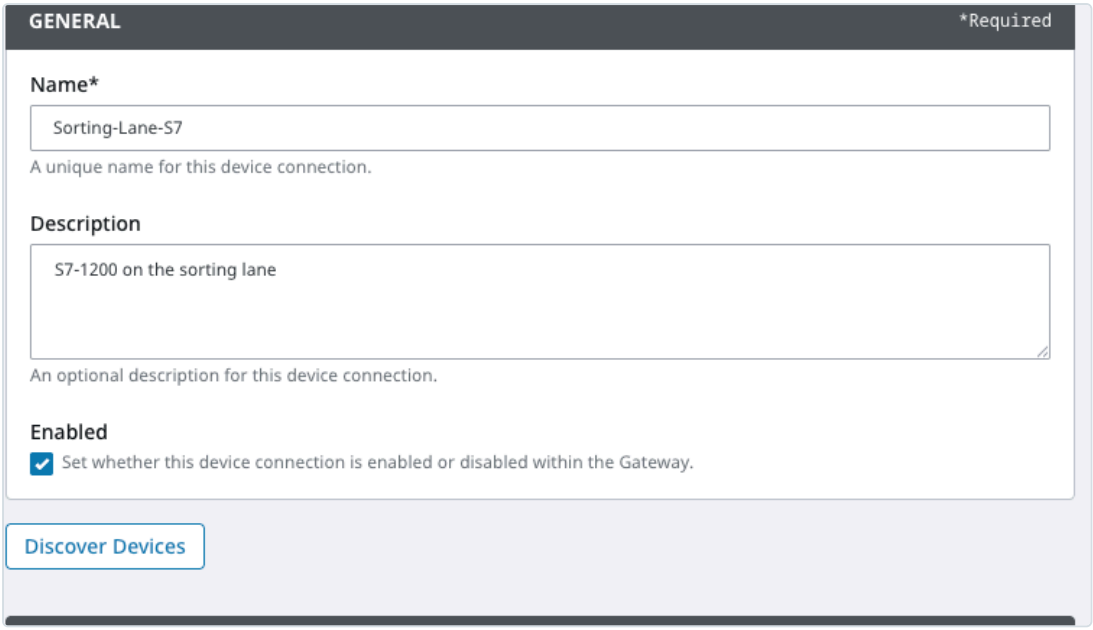
**Security.** The agent listens only on the Docker networks' own addresses on that machine (port 47900), never on a plant network, and accepts connections only from the container networks. To restrict it further, give it a shared secret: `-e TODDYSOFT_AGENT_SECRET=<secret>` on the agent and `-e TODDYSOFT_DISCOVERY_AGENT_SECRET=<secret>` on the Ignition container. With a secret set, a gateway without it, or with another, is refused. To stop the module looking for an agent, set `TODDYSOFT_DISCOVERY_AGENT=off` on the Ignition container.

## Configuration

The configuration form is organized into the following sections, top to bottom: **General**, **Transport**, **S7**, **TLS**, **Authentication**, **Limits** and **Audit Log**.

### General

Assigns a name (and optional description) to the connection and controls whether it is enabled.




| Setting | Description | Default |
|---|---|---|
| **Name** | Unique name for this device connection | |
| **Description** | Optional description for this connection | |
| **Enabled** | Whether the connection is active | `true` |

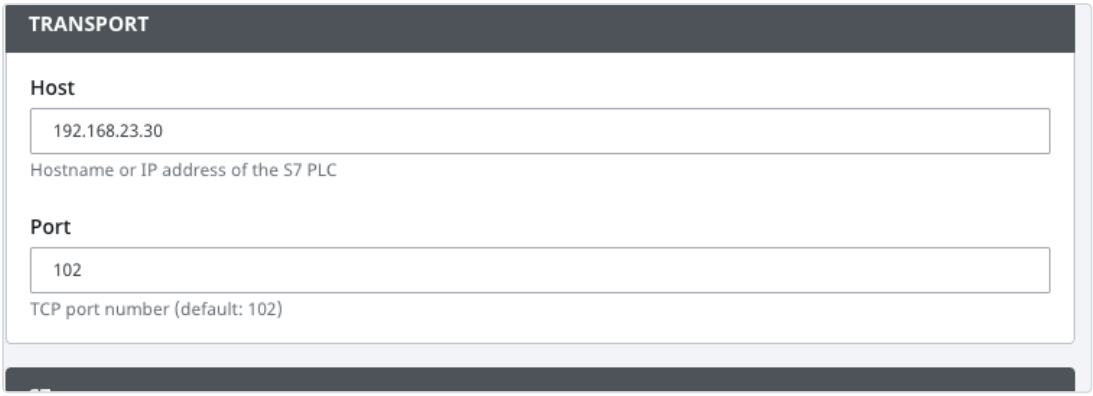
This is the name the connection is then referenced under in other parts of Ignition.

If `Enabled` is not checked, no connection is established until it is enabled.

## Transport

| Setting | Description | Default |
|---|---|---|
| **Host** | Hostname or IP address of the S7 PLC | `localhost` |
| **Port** | TCP port number | `102` |

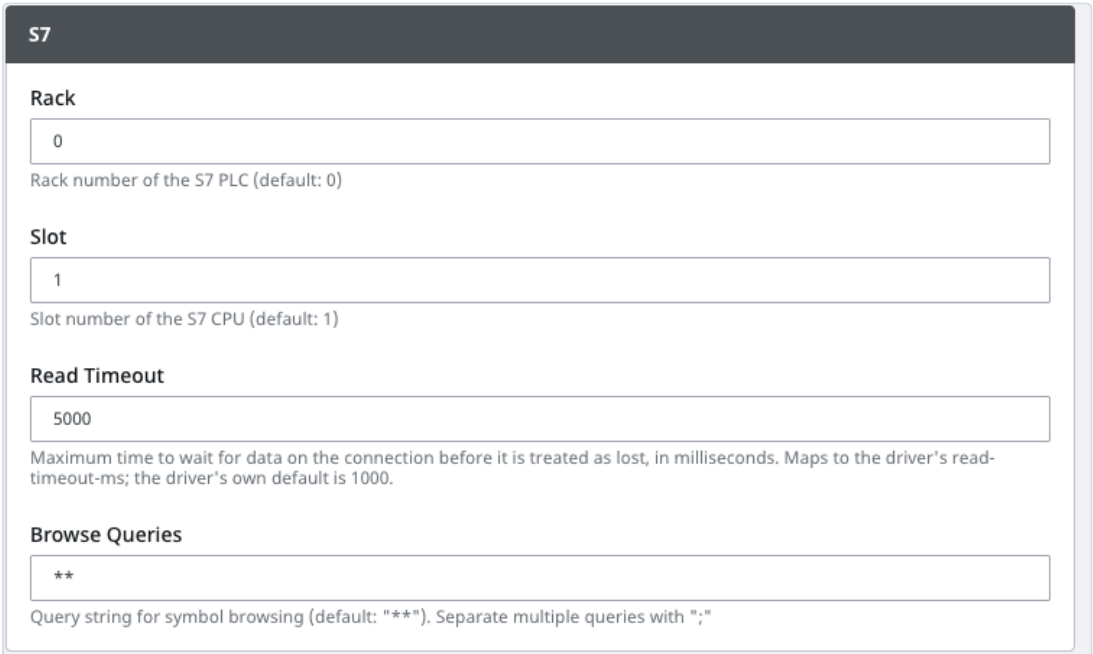
There is no transport selector on this driver, and none is missing. S7 speaks exactly one transport — `cotp`, ISO-on-TCP per RFC 1006 — and **TLS here is not a transport**: the S7CommPlus handshake runs *inside* COTP framing on the same port 102. There is no second port and no separate scheme to choose. Whether the session ends up encrypted is decided by the **TLS** section below, not here.




## S7

| Setting | Description | Default |
|---|---|---|
| **Rack** | Rack number of the S7 PLC | `0` |
| **Slot** | Slot number of the S7 CPU | `1` |
| **Read Timeout** | Maximum time to wait for data on the connection before it is treated as lost, in milliseconds | `5000` |
| **Browse Queries** | Symbol browse query filter. Separate multiple queries with `;` | `**` |

**Rack and slot are worth double-checking.** They identify *which CPU* in the rack the session addresses, and a wrong pair frequently connects — to a different CPU, which then answers reads with perfectly valid data from the wrong machine. The usual values are rack 0 / slot 1 for an S7-1200 or S7-1500, and rack 0 / slot 2 for a classic S7-300 / S7-400. **Discover Devices** reads them off the controller, which is the reliable way to get them.



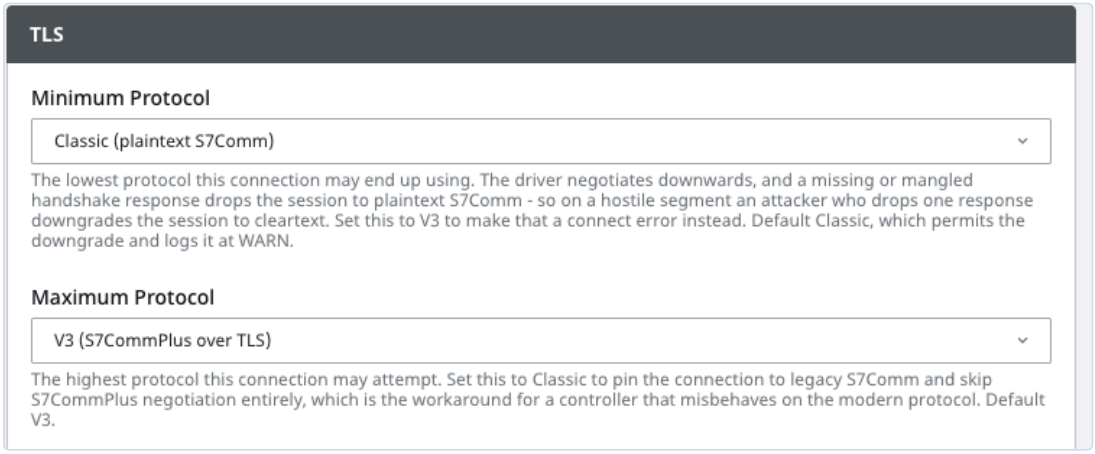
**Read Timeout** is how long the driver waits for data on an established connection before treating the link as lost. It is not a per-request budget: a slow controller under load does not need this raised, a flaky network link does.

## TLS

S7CommPlus can run the session inside TLS, and on an S7-1200 / S7-1500 it normally does. These settings decide whether that happens and what the driver accepts on the other end.

| Setting | Description | Default |
|---|---|---|
| **Minimum Protocol** | The lowest protocol this connection may end up using | `Classic (plaintext S7Comm)` |
| **Maximum Protocol** | The highest protocol this connection may attempt | `V3 (S7CommPlus over TLS)` |
| **Verify Certificate** | Validate the controller's TLS certificate | `true` |
| **Pinned Certificate Fingerprints** | Comma-separated SHA-256 fingerprints of the leaf certificates to accept (hex, colons optional) | *(empty)* |
| **Trust Store Path** | JKS or PKCS12 trust store holding the CA that signed the controller certificate | *(empty)* |
| **Trust Store Password** | Password for the trust store. Leave as *None* for an unprotected store | *None* |
| **TLS Version** | Comma-separated allow-list of TLS versions to offer | `TLSv1.3` |

### The protocol window, and the silent downgrade

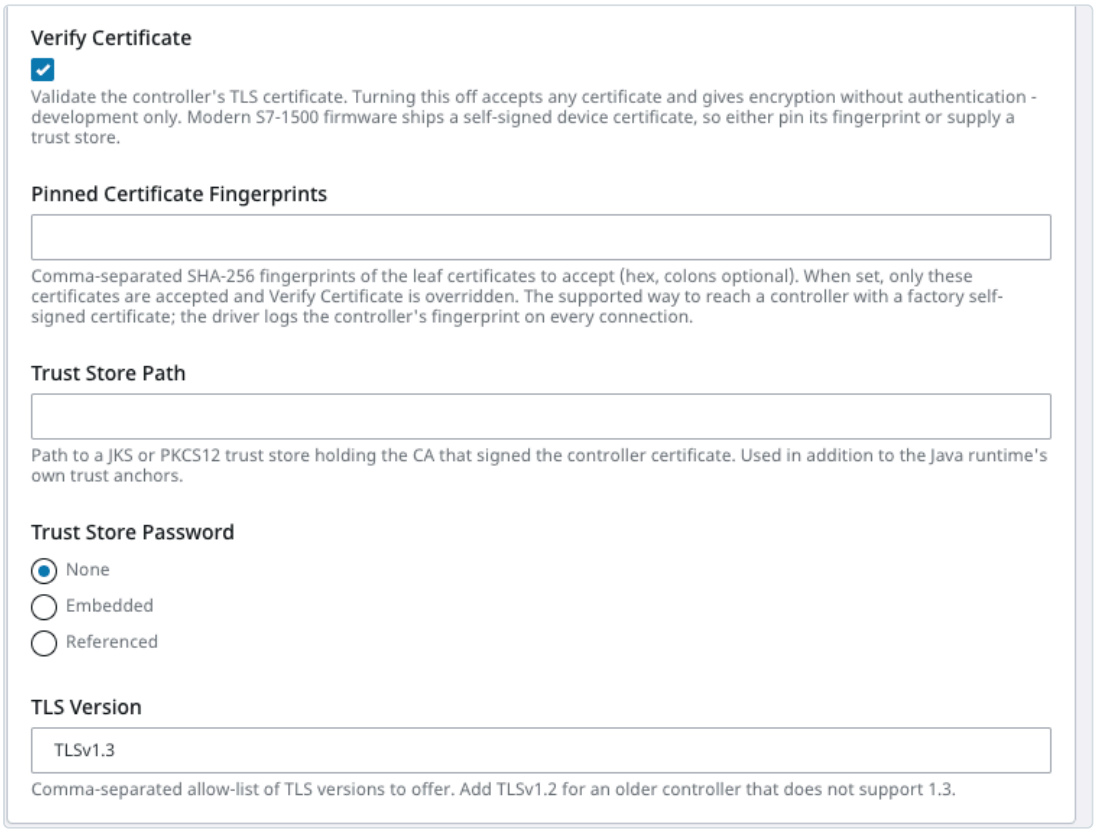


The driver negotiates **downwards**. If the controller's handshake response is missing or mangled, the session drops to plaintext S7Comm and carries on working — the tags read, the device shows *Connected*, and nothing announces that the encryption is gone. On a hostile network segment, an attacker who drops a single response downgrades the session to cleartext.

That default exists because it is the behaviour that connects to the widest range of hardware, and the downgrade is logged at `WARN`. **On any connection you consider security-relevant, set *Minimum Protocol* to `V3 (S7CommPlus over TLS)`.** The downgrade then becomes a connect error instead of a warning nobody reads.

*Maximum Protocol* works the other way: setting it to `Classic (plaintext S7Comm)` pins the connection to legacy S7Comm and skips S7CommPlus negotiation entirely. That is the workaround for a controller that misbehaves on the modern protocol, not a setting to reach for casually.

### Trusting the controller's certificate




Modern S7-1500 firmware ships a **self-signed device certificate**. It is not signed by any CA, so ordinary verification cannot succeed against it, and there are exactly two honest ways forward:

1. **Pin the fingerprint** (recommended). Paste the controller's SHA-256 fingerprint into **Pinned Certificate Fingerprints**. When set, only those certificates are accepted and *Verify Certificate* is overridden. The driver logs the controller's fingerprint on

every connection, so you never have to compute it by hand. This is secure *and* it works out of the box.
2. **Supply a trust store**, if the certificate was signed by your own CA. The store is used in addition to the Java runtime's own trust anchors.

Turning **Verify Certificate** off gives you encryption without authentication — anyone who can intercept the connection can terminate the TLS themselves. It is a development setting. Pin a fingerprint instead; it is no more work.

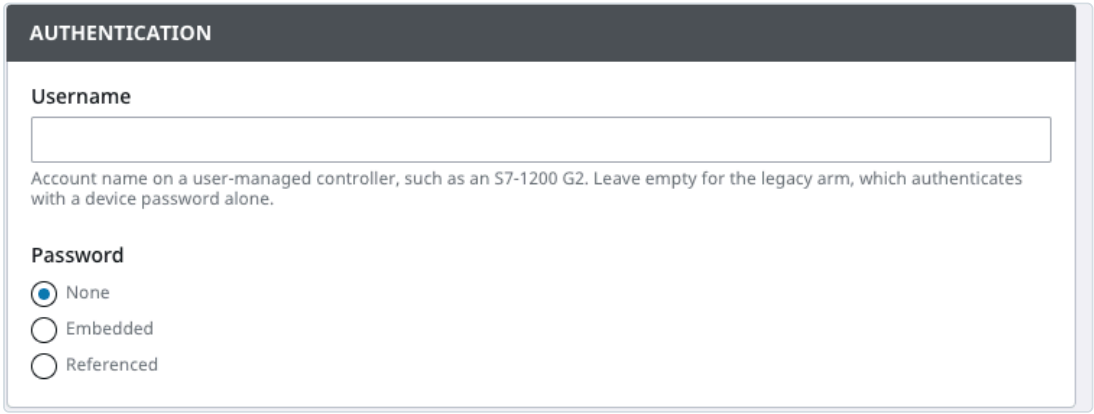
**TLS Version** defaults to `TLSv1.3`. Add `TLSv1.2` for an older controller that does not support 1.3 — the field is an allow-list, so `TLSv1.3,TLSv1.2` offers both.

## Authentication

S7CommPlus **legitimation** — the driver's only authentication method, and a protocol-layer one. It runs inside the S7CommPlus conversation once the connection is up, so it is unrelated to the TLS settings above: a plaintext session can be legitimated and an encrypted one need not be.

| Setting | Description | Default |
|---|---|---|
| **Username** | Account name on a user-managed controller, such as an S7-1200 G2. Leave empty for the legacy arm, which authenticates with a device password alone | *(empty)* |
| **Password** | Controller password. Leave as *None* for a controller that does not require legitimation | *None* |

Whether you need this at all depends on the controller's **protection level**, which is set in the TIA project and is not knowable until the driver connects. A controller that does not demand legitimation connects with no credential, which is the common case — leave both fields alone.



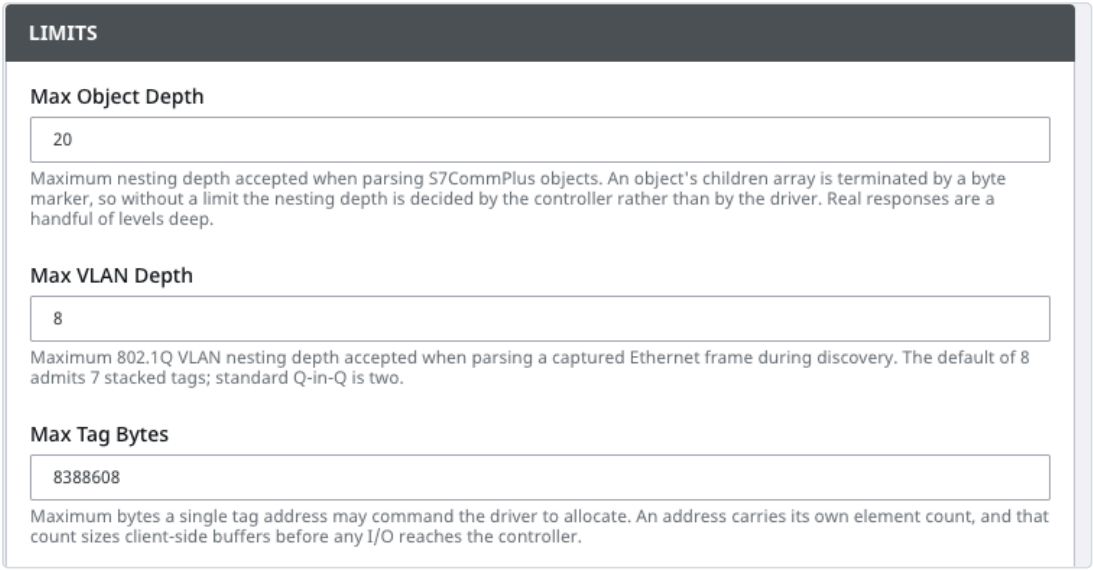

**The password is what authenticates.** A username alone legitimates nothing (both arms present a password), so nothing is sent unless a password is set. There is no arm selector: the driver picks between the legacy and named-user mechanisms from the

controller's article number and firmware, and supplying a username is itself enough to select the named-user arm.

Since ToddySoft Connect 1.2.0 the credential is handed to the driver as an object rather than written into the connection string, so it does not appear in the gateway log, in a support bundle, or in any string quoting the connection.

## Limits

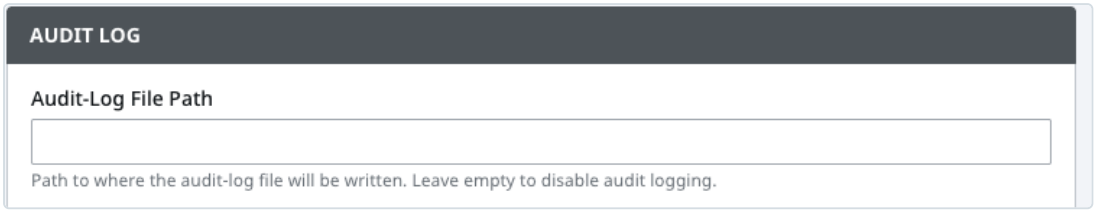
Ceilings on what the controller can make the driver allocate. Each one bounds something the *other* side chooses, so a device that misbehaves — or a malformed frame on the wire — cannot exhaust the gateway. The defaults are the driver's own; raise one only for a controller measured to need it. Leaving them alone produces exactly the connection the device made before these settings existed.




| Setting | Description | Default |
|---|---|---|
| **Max Object Depth** | Maximum nesting depth accepted when parsing S7CommPlus objects. An object's children array is terminated by a byte marker, so without a limit the nesting depth is decided by the controller rather than by the driver. Real responses are a handful of levels deep. | `20` |
| **Max VLAN Depth** | Maximum 802.1Q VLAN nesting depth accepted when parsing a captured Ethernet frame during discovery. Discovery frames arrive as unauthenticated broadcast, at four bytes per level. The default of 8 admits 7 stacked tags; standard Q-in-Q is two. | `8` |
| **Max Tag Bytes** | Maximum bytes a single tag address may command the driver to allocate. An address carries its own element count, and that count sizes client-side buffers before any I/O reaches the controller. | `8388608` |

### Audit Log

The `Audit Log` writes important diagnostic information to a dedicated log file. This file can grow quickly, so it is advised to only enable it when investigating issues.




| Setting | Description | Default |
|---|---|---|
| **Audit-Log File Path** | Absolute path to the audit log file where events will be recorded. Leave empty to disable. | *(empty)* |

NOTE: Please ensure the user the `Ignition` process is running under has write access to this path.

The Audit Log system automatically creates the file and rotates it (creating compressed files next to it) once a certain size is reached or on a time-based schedule.

## Tag Addressing

This driver accepts two address forms. They are not alternatives to pick between: the absolute form works everywhere, the symbolic form works only on S7-1200 / S7-1500 via S7CommPlus.

### Absolute addresses (all controllers)

Siemens' own absolute-address style:

```
%<area><offset>[.<bit>]:<datatype>[(<size>)][[<selection>]]
area     ::= DB<n> | M | I | Q | T | C
```

| Address | Meaning |
|---|---|
| `%DB1:0:INT` | Signed 16-bit int at byte 0 of DB1 |
| `%DB1:4.0:BOOL` | Bit 0 of byte 4 of DB1 |
| `%DB1:10:STRING(20)` | S7 STRING (length-prefixed, up to 20 chars) at byte 10 of DB1 |
| `%DB1:0[0..9]:REAL` | Ten floats starting at byte 0 of DB1 — a range, inclusive of both bounds |

| Address | Meaning |
|---|---|
| `%DB1:0[10]:REAL` | **One** float, the eleventh (index 10). A bare index is a single element, not a count |
| `%DB1:0[4..7;1]:REAL` | Elements 4–7 of an array TIA declares as `Array[1..10]` |
| `%DB1:0[0..1][0..2]:INT` | A 2×3 block of ints. One bracket per dimension; `[0..1,0..2]` is the same selection |
| `%M0.0:BOOL` | Bit 0 of M0 (merker bit) |
| `%I0:BYTE` | Input byte 0 |
| `%Q0.5:BOOL` | Bit 5 of Q0 (output bit) |

The longer `%DB<n>.DBX<byte>.<bit>:<datatype>` form is also accepted. Addresses typically come from the TIA Portal symbol export.

> **An array selection is written *before* the `:TYPE` suffix** — `%DB1:0[0..9]:REAL`, not `%DB1:0:REAL[0..9]`. The old position no longer parses, and the error names the replacement. The parenthesis is reserved for a string's declared length and means nothing else.

## Symbolic addresses (S7-1200 / S7-1500)

On an S7-1200 or S7-1500 a tag can be addressed by name, in either of two equivalent forms:

| Form | Example | Notes |
|---|---|---|
| Block name, quoted | `"MyDataBlock".g_plant.channels[2].setpoints[3]` | TIA Portal's own display form — copy-pasteable straight out of TIA. This is what browsing produces. |

| Form | Example | Notes |
| --- | --- | --- |
| Block number | `DB43.g_plant.channels[2].setpoints[3]` | Always valid. Used when a block's name is missing or not unique. |

Both resolve to the same tag, and addresses already written against the number form keep working.

**Why the block name is quoted.** A data block may legitimately be *named* `DB42`, so a bare `MyDataBlock.myVar` could not be told apart from the number form without knowing every block name on the controller. The quotes also let a block name contain spaces or dots without escaping — `"OPC Data".myVar`, `"Plant.Line1".speed`.

**An ambiguous name is refused, not guessed.** The name comes from the controller's type-name attribute, so two instance data blocks of the same function block can report the same string. Addressing such a name directly fails with a message naming the colliding blocks, rather than resolving to whichever was loaded first and returning wrong values from a read reporting success.

**A block or memory-area node is a container, not an address.** In the browse tree a data block (`"MyDataBlock"` / `DB42`) and a memory area (`%M`, `%I`, `%Q`) carry no tag and report neither readable nor writable. Read its children, not the node itself. `%DB42:0:BYTE` remains a perfectly valid hand-written address — only the browse tree stopped offering it as a stand-in for a whole block.

Configure **Browse Queries** (e.g. `DB1.*`) to control which symbols appear when browsing tags in the Ignition Designer.

## Tag Groups

By default, all tags are collected in the default tag group, which means all selected tags are fetched at the default interval of once a second.

If different intervals are desired, create additional tag groups, configure the desired intervals there, and assign tags to the corresponding tag groups.

## Licence diagnosis

When a device reports a licence problem, the module explains it on the gateway itself. Open the device's settings (**Config > OPC UA > Device Connections**, edit the device, or create a new one): the **Licence** card at the top of the form shows the diagnosis. It is

on the form for a new device too, so the licence can be checked before the first device exists.

The diagnosis shows:

- **which gateway** this is, and its role if it is part of a redundant pair;
- **Ignition's licence** for this module on this gateway: activated, trial running (with its end) or trial expired;
- **what ToddySoft licence data arrived** with Ignition's licence: how many parts, whether they are complete, and a checksum that identifies the licence without revealing it;
- the **driver's own verdict** and the **cause**, in words.

| Cause | Meaning | What to do |
|---|---|---|
| `IGNITION_UNLICENSED` | Ignition applies no activated licence to this module on this gateway (trial running or expired). ToddySoft licence data only arrives with an activated licence. | Activate a licence key that covers this module on this gateway. In a redundant pair every node needs its own activation. If a key was activated and this cause remains, see Troubleshooting: an older, expired key covering this module may be the one Ignition judges it by. |
| `NO_DATA_FOR_DRIVER` | The activated key carries no ToddySoft licence for this driver. | Send the report to ToddySoft. |
| `DATA_INCOMPLETE` | The licence data arrived incomplete or damaged. | Send the report to ToddySoft; the licence has to be delivered again. |
| `DATA_REJECTED` | The driver refused the delivered licence; the reason is shown. | Send the report to ToddySoft. |
| `RUNNING_ON_BUNDLED_TRIAL` | A licence was delivered but not accepted, and the driver runs on its built-in trial until that ends. | Send the report to ToddySoft. |

| Cause | Meaning | What to do |
| --- | --- | --- |
| `NOT_CHECKED` | Licence data arrived, but the driver gave no verdict - for instance because it could not be loaded. | Send the report to ToddySoft. |
| `VALID` | Licensed; tier and expiry are shown. | Nothing. |
| `NOT_YET_KNOWN` | The first seconds after start-up, before Ignition reports a licence state. | Wait a moment. |

**Sending a report to support.** The diagnosis comes as a plain-text report, the one thing to send with any licence question. **Copy report** puts it on the clipboard; **Show report** displays it. The same text is served at `http://<gateway>:<port>/data/com.toddysoft.connect.s7/licence-report` to a logged-in user. It contains no licence data, no keys, no credentials and no customer details. In a redundant pair, send the report from both nodes.

The gateway log carries the same report once each time the licence situation changes.

## Troubleshooting

**Licensing problems after a new or renewed licence key was activated — `DATA_INCOMPLETE` with "duplicate fragment index", or `IGNITION_UNLICENSED` with "trial expired", in the licence diagnosis**:

- **Two licence keys covering this module are activated on the gateway at the same time**, typically a renewed or upgraded key activated alongside the old one. Ignition hands the module the licence data of both, in numbered parts that carry the same numbers; the module cannot tell which parts belong together, keeps one of each pair, and the licence it reassembles is unreadable. Depending on the parts it keeps, devices are refused or run on the module's bundled trial until that ends.
- If one of the two keys has **expired**, Ignition may judge the module by that key alone: the diagnosis shows `IGNITION_UNLICENSED` ("trial expired - no activated licence applies to this module") and "Data: none received", although the new key was activated successfully.
- Keep exactly **one** activated licence key that covers this module per gateway: deactivate the old one under **Platform > Licensing**, then restart the module (or the

gateway). In a redundant pair, check every node.

**The module will not load, or the log shows `ClassNotFoundException` / `NoClassDefFoundError`**:

- Ignition's **OPC UA** module is missing or disabled. This driver is an OPC UA device driver and cannot load without it. Install or re-enable it under **Config > System > Modules** and restart the gateway — see Installation

**Device shows "Faulted"**:

- Verify the PLC hostname/IP is reachable
- Check that **Rack** and **Slot** match the physical configuration — `0` / `1` for S7-1200 and S7-1500, `0` / `2` for a classic S7-300 / S7-400
- Ensure port 102 is not blocked by a firewall
- Review Gateway logs: **Status > Diagnostics > Logs**

**The device connects, but the values are from the wrong machine**:

- Almost always a wrong **Rack** / **Slot** pair addressing a different CPU in the same rack. It connects, and it reads. Use **Discover Devices** to read the real values off the controller.

**Tags show bad quality**:

- Verify the variable exists in the PLC program
- Check spelling and case sensitivity; a quoted block name must match TIA exactly
- Ensure the PLC program is running
- An absolute address written with the array selection after the type — `%DB1:0:REAL[0..9]` — no longer parses; move the selection before the type

**Writes are refused although reads work**:

- The controller's **protection level**, set in the TIA project, governs writes separately from reads. Supply the controller password under **Authentication**, or lower the protection level in TIA.

**The connection is not encrypted although the controller supports TLS**:

- The driver negotiates downwards and a missing or mangled handshake response drops the session to plaintext S7Comm, logging a `WARN`. Set **Minimum Protocol** to `V3 (S7CommPlus over TLS)` to turn that into a connect error. See *The protocol window, and the silent downgrade*.

**TLS connection fails**:

- A factory self-signed device certificate cannot pass ordinary verification. Pin its fingerprint under **Pinned Certificate Fingerprints** — the driver logs it on every connection

- For a certificate signed by your own CA, set **Trust Store Path**
- For an older controller, add `TLSv1.2` to **TLS Version**

### Docker related issues

**Device discovery needs the discovery agent.** *Discover Devices* searches the networks the gateway itself is attached to, which for a container are Docker's, not the plant's. Run the ToddySoft discovery agent next to the container — see Running in Docker: the discovery agent — or enter the host, rack and slot by hand.

Ordinary connections are unaffected: unlike ADS, the S7 protocol carries no identity for this end of the connection, so a container's internal address is never announced to the controller and nothing has to be reconciled.

### VPN related issues

A remote controller reached over a VPN needs nothing special from this driver, for the same reason: there is no source identity to configure. If the connection fails, it is ordinary reachability — confirm that port 102 is routed and not blocked.

### Several devices on the same PLC

Several Ignition devices may address the same controller at once; there is no per-gateway identity to collide, as there is with ADS. What is limited is the controller itself: an S7-300 or S7-400 has a small fixed number of connection resources, and once they are exhausted further connects are refused until one is released. An S7-1200 / S7-1500 has considerably more, but not an unlimited number.

If devices on the same controller connect and drop in rotation, count the connections the controller is serving — including TIA Portal sessions and HMIs — before looking for a fault in the module.

## Version history

### Unreleased — 1.3.0

Built on ToddySoft Connect 1.3.0.

- **Added:** this manual.
- **New:** licence diagnosis — why a device is or is not licensed, shown on the gateway, with a report to send to support. See Licence diagnosis.
- **New:** troubleshooting for two licence keys covering this module activated on one gateway at the same time. See Troubleshooting.
- **New:** discovery when Ignition runs in Docker, through the ToddySoft discovery agent. See Running in Docker: the discovery agent.

- **Changed:** date and time tags have new value types. DATE and DATE_AND_TIME are DateTime values (until now a DateTime tag holding epoch milliseconds as a number); TIME and LTIME are milliseconds, TIME_OF_DAY and LTIME_OF_DAY milliseconds since midnight (Int64). Writes take the same forms. Adjust scripts and bindings that used the old values.
- **Fixed:** a write the PLC refuses now fails in Ignition, with the reason as the status (for example `Bad_TypeMismatch` for a value of the wrong type). Until now every write was reported as successful, and the refusal appeared only in the gateway log.
- **Fixed:** a tag that has not been read yet, or whose last read failed, no longer reads 0 with Good quality. Before its first read it has no value and the quality Uncertain; after a failed read it keeps its last value with Bad quality until the next good read.
- **Fixed:** removing the last tag of a tag group (or polling rate) no longer leaves the device showing *Connection Lost*. A poll still running at that moment made the device fail with `Batch '...' not found` in the gateway log and stay in that state until it was restarted.
- **Changed:** a browsed array is one folder, named after the array and its index range (for example `arr[0..9]`), holding its elements; until now the elements sat directly beside the array's neighbours. A multi-dimensional array, until now left out of the browse, appears as nested folders down to its elements (`cube[1..2][1..2]`, then `cube[1][1..2]`, then `cube[1][1]`). An array of structures shows a folder per element holding its members (for example `pts[2].x`). Each element keeps its OPC item path, so existing tags keep working.